

Pipelining FALCON’s Gaussian sampler

Towards practical FALCON acceleration

Michal Vojáček and Martin Novotný

CTU in Prague, Prague, Czech Republic, {vojacmi3,novotnym}@cvut.cz

Abstract. We present the first fully pipelined implementation of FALCON’s Gaussian sampler `SamplerZ`, which has previously dominated signature-generation time. The design mitigates the latency cost of rejection sampling by launching a new sampling attempt every clock cycle. On the ZCU104 FPGA, our accelerator achieves a Rejection-Inclusive Pair Sampling Time (RIPST) of 100.0 ns, a 5.6x speedup over the best prior area-efficient design. At the same time, it reduces area across all major FPGA resource classes and requires only 30 DSPs, corresponding to a 5.6x/6.7x/17.7x/9.4x (LUT/FF/DSP/BRAM) RIPST-area-product improvement over the strongest prior baseline. Beyond the pipelined architecture itself, we introduce algorithmic and implementation-level optimizations that reduce latency, improve portability, and significantly lower randomness consumption. The resulting design is compact, portable, and reproducible, and provides a practical foundation for future FALCON signing implementations.

Keywords: Post-quantum cryptography · FALCON · discrete Gaussian sampling · `SamplerZ` · rejection sampling · accelerator · low latency · compact · FPGA · pipeline

1 Introduction

Traditional public-key cryptosystems such as RSA and ECC rely on computational problems believed to be hard for classical computers, such as integer factorization and the discrete logarithm. Given a quantum computer with sufficient qubits and a low error rate, these systems can be broken in polynomial time using Shor’s algorithm [Sho94]. Because quantum computing has advanced considerably in recent years, the National Institute of Standards and Technology (NIST) recommends transitioning away from classical cryptography as soon as 2035 [MPR⁺24], motivating the development of new post-quantum cryptography (PQC) algorithms. To this end, in 2016, NIST launched the PQC Project [Nat17], selecting four algorithms for standardization in 2022. Among them, a promising candidate for use in TLS [SKD20] is FALCON [PFH⁺22], a lattice-based signature scheme with compact signatures and fast verification. However, FALCON’s signature-generation procedure is complex and resource-intensive. At its core lies discrete Gaussian sampling via `SamplerZ` [PRR19], which still accounts for up to 65 % of the signing time even in recent high-efficiency implementations with a dedicated `SamplerZ` accelerator [OZZ⁺25]. Recent work has therefore focused on practical low-latency hardware `SamplerZ` accelerators to remove this bottleneck.

Related works. The first `SamplerZ` accelerator, a HW/SW co-design by [KA23], used a custom pipelined DSP multiplier to generate one sample in 1807 cycles. [SAW⁺23] provided the first pure-hardware implementation of the full FALCON signature by applying High-Level Synthesis (HLS) directly to the reference C implementation, albeit at considerable area and latency cost. [YSZ⁺24] introduced RVCE-FAL, a compact RISC-V

instruction-set extension built around a single fused multiply-sub floating-point core, achieving a latency of 104 cycles per sample. [LYN⁺24], the first ASIC-only implementation, proposed decomposing the algorithm into common operations and dispatching them onto a set of shared operation blocks.

To improve throughput, [OZZ⁺25] designed a three-part high-level pipeline in their “FalconSign” accelerator, reducing latency to 67 cycles for the first sample and 40 cycles for each subsequent sample. The authors of [ZASM25] further optimized this pipeline approach to 59 cycles and 34 cycles, respectively, for a *pair* of samples. The current state-of-the-art implementation by [PMKA25] utilizes proprietary floating-point IPs to achieve a latency of 22 cycles per sample.

However, rejection sampling is the main obstacle to a fair latency comparison. Since **SamplerZ** is probabilistic (see Algorithm 1), sampling may fail and require re-computation. In all prior works, a rejection requires at least a partial restart of the pipeline, introducing significant and data-dependent latency. For instance, [ZASM25] notes that rejections nearly double their average latency, from 59 to 106 cycles. Some earlier works did not include this effect in their reported figures at all [KA23, YSZ⁺24, OZZ⁺25], making direct comparisons difficult. [ZASM25] reports rejection-inclusive figures derived from simulation. While [PMKA25] introduced the Rejection-Adjusted Execution Time (RAET) framework for average latency, due to some shortcomings, we define the Rejection-Inclusive Pair Sampling Time (RIPST) metric instead.

2 Our contributions

Optimized floating-point floor. In previous implementations, computing the floating-point floor has been expensive in both area and latency. We present a hand-optimized module specifically for **SamplerZ** that requires $5\times$ fewer cycles, reaches a higher maximum frequency, and consumes $3\times$ fewer LUTs than [OZZ⁺25], while requiring no FFs or DSPs.

Uniform comparison randomness usage reduction. The **BerExp** algorithm (Algorithm 2) compares a computed 64-bit number to random bits obtained from `UniformBits(64)`. In the FALCON specification [PFH⁺22], this comparison is carried out byte-by-byte from the most significant byte, so only close to 8 bits are required on average. We propose a *refillable uniform comparator*, which enables bit-by-bit comparison in hardware while balancing performance and area, reducing the average consumption to **2.1885 bits** per 64-bit comparison ($3.67\times$ reduction compared to specification).

Boolean Knuth-Yao BaseSampler. Using an improved approach compared to previous hardware implementations, we implement **BaseSampler** (Algorithm 3) with $3\times$ fewer resources. Moreover, an extended version of this **BaseSampler** implementation can report the number of consumed random bits, making it compatible with the same randomness-consumption reduction technique as the uniform comparator and reducing the average required number of random bits per sample from 72 to just **3.52** ($20.45\times$ reduction compared to specification) at a slight increase in area.

Floating-point exponentiation using FloPoCo. Instead of implementing the software-oriented algorithm from the specification, we implement **BerExp** using a configurable, open-source, platform-agnostic, pipelined exponential IP core [dDP10] from the FloPoCo suite [dD19]. In our setting, this reduces the isolated exponentiation latency to 36.0 ns at 222.2 MHz while using only 6 DSPs and 1.5 BRAMs, giving a clear latency and DSP advantage over traditional implementations [OZZ⁺25, ZASM25] and a portability advantage over the recent Xilinx-IP-based design of [PMKA25].

Parallel comparison with *ccs*. We eliminate one FP64 multiplier and its associated 2 cycles of latency in **BerExp** by comparing *ccs* with a second invocation of **UniformBits**(64) instead of multiplying by *ccs*. Although this doubles randomness consumption, when combined with our uniform-comparison improvements, the actual average usage of randomness in one **BerExp** invocation is still less than **4.377 bits** (minimum **1.83×** reduction compared to specification).

Custom integer-FP conversion modules. We wrote a custom module for the conversion of small integers (specifically z and z_0^2 in **SamplerZ**) to IEEE754 FP64. We also provide a custom implementation of the $\lfloor 2^{64} \cdot p \rfloor$ operation in **BerExp**. Along with the *fractional decomposition* module, this eliminates the need for any third-party integer-FP conversion cores, improving portability, latency, and compactness.

SamplerZ pipeline. We implemented **SamplerZ** as one continuous 19-stage pipeline, yielding a time-to-first-sample (TTFS) latency of 85.6 ns, compared to 163 ns (**1.9×**) in [PMKA25] and 362 ns (**4.2×**) in [OZZ+25]. Because the pipeline accepts a new attempt every cycle, the time-to-next-sample (TTNS) is just 1 cycle or 4.5 ns, compared to 163 ns (**36×**) in [PMKA25] and 216 ns (**48×**) in [OZZ+25]. To the best of our knowledge, this is the first **SamplerZ** accelerator to reduce the cost of a rejection to the theoretical minimum of one cycle per reattempt.

Alternating pipeline sampling. As both samples in one **ffSampling** pair (Algorithm 4) can be computed concurrently, they are injected into the **SamplerZ** pipeline in alternation, with shared σ -derived computations scheduled onto otherwise idle pipeline resources only once. In this arrangement, both samples succeed after 4.21 attempts on average, which translates to **22.2 cycles** or **100 ns** total for two samples on a single module (RIPST).

For comparison, an arrangement of two parallel instances of our **SamplerZ** module would obtain both samples in just 20.2 cycles (**1.1×** faster), but at **2×** the area (**1.8×** worse time-area product), while serial computation on a single module would take 39.5 cycles (**1.8×** slower), proving the time-area superiority of the alternating pipeline approach.

Continuous sampling mode. While not directly useful for signature generation, our **SamplerZ** pipeline can also run indefinitely with a single parameter set, producing one sample every 1.737 cycles at 222 MHz. Paired with a hardware histogram, this enables very large datasets to be collected in reasonable time and supports high-confidence statistical validation of the output distribution. On a single module instance, we collected 2.6×10^{13} samples in ≈ 2 days and verified the distribution against theoretical expectations. The same experiment would take ≈ 47 days on [PMKA25]’s C3 design and ≈ 74 days on their more area-comparable C1 design.

Reproducibility and documentation. Following the open approach of recent implementations [OZZ+25, ZASM25, KA23, SAW+23], the full source code of our implementation is available¹. We also provide project-creation and implementation scripts for Vivado 2025.1 targeting the ZCU104 and Kria KV260 boards, and the flow can be extended to other Xilinx/AMD boards with little effort.

To port the accelerator to a non-AMD platform, only two proprietary IP blocks must be replaced: FP64 multiplication and subtraction. Both are instantiated only through SystemVerilog wrappers, so each can be replaced at a single integration point. Because the latencies of all IP cores, including the open-source exponential, are encoded as SystemVerilog

¹<https://github.com/mvojacek/ches-2026-falcon-samplerz>

constants and the pipeline adapts to those values, the design can accommodate any non-blocking pipelined replacement with arbitrary latency. In principle, this enables portability across vendors and technologies, as well as straightforward retiming for different frequency targets.

3 Background

3.1 Notation and conventions

The sets of complex numbers, real numbers, rationals, integers, and non-negative integers are denoted $\mathbb{C}, \mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$, respectively. The symbol $\mathbb{Z}^+$ (resp. $\mathbb{Z}^-$) denotes the set of non-negative (resp. non-positive) integers, and similarly for $\mathbb{R}$. The notation $z \sim \mathcal{P}$ means that z is sampled from distribution $\mathcal{P}$ with probability mass function (PMF) $p(z)$. The $\text{sgn}_*(x)$ function is the signum function with the special convention that $\text{sgn}_*(0) = 1$. The expression $\llbracket \text{condition} \rrbracket$ is 1 precisely when *condition* is true, and 0 otherwise. FP stands for floating-point numbers represented in IEEE754 format, and FP64 for 64-bit FP.

3.2 FALCON Signature Scheme

FALCON is a practical instantiation of the GPV scheme [GPV08], which is itself the secure variant of the GGH scheme [GGH97], over 512- or 1024-dimensional NTRU lattices [HHP⁺03]. The security of these schemes relies on the hardness of the Short Integer Solution (SIS) problem [Ajt96] on high-dimensional lattices: finding the coordinates of a lattice point close to zero with specific properties. In this scheme, generating signatures means finding such a point quickly using secret information about the lattice (the private key).

In the GGH scheme, orthogonalization and rounding are used to quickly find a very short solution, but it was later shown that the private key could be recovered from as little as 90,000 signatures [NR06] due to the exactness of the rounding process. The GPV scheme randomizes the rounding using a *trapdoor sampler*, producing slightly worse (longer) signatures, but preventing the private-key recovery attack [PFH⁺22].

3.2.1 FALCON’s trapdoor sampler

A *trapdoor sampler* returns a point close to the ideal signature, sampled from a many-dimensional discrete Gaussian distribution over the lattice, by combining many one-dimensional discrete Gaussian samples. The first proposed trapdoor sampler by Klein [Kle00] suffered from a high asymptotic time complexity with respect to the lattice dimension, rendering it impractical.

“Fast Fourier Orthogonalization” [DP15], implemented in the **ffSampling** (Algorithm 4) algorithm, was proposed as a faster alternative. The speed-up comes from exploiting a tower structure in NTRU lattices with a divide-and-conquer approach reminiscent of the Fast Fourier Transform². The **ffSampling** algorithm recursively traverses a binary tree in depth-first order, reducing the problem dimension until it reaches one-dimensional discrete Gaussian sampling from $D_{\mathbb{Z}, \sigma, \mu}$, where $\sigma, \mu \in \mathbb{R}$ vary between samples. This final sampling step is itself nontrivial and is implemented by the **SamplerZ** algorithm.

For a single FALCON-512 (resp. 1024) signature, **SamplerZ** must be invoked 1024 (resp. 2048) times. Fortunately, each pair of adjacent samples can be parallelized (Algorithm 4) and even shares the same σ [OZZ⁺25], but the 512 (resp. 1024) pairs must be computed sequentially. No further calculations are possible until both samples in the pair are obtained,

²Additionally, all calculations are performed in the Fourier (frequency) domain for another speed-up, but this is unrelated to the “Fourier-like structure” of the **ffSampling** algorithm itself.

Algorithm 1: SamplerZ(μ, σ)

Input: $\mu, \sigma \in \mathbb{Q}$, such that $\sigma \in [\sigma_{\min}, \sigma_{\max}]$
Output: $z' \in \mathbb{Z}$ sampled from a distribution very close to $D_{\mathbb{Z}, \sigma, \mu}$

```

1  $r \leftarrow \mu - \lfloor \mu \rfloor$   $\triangleright$  fractional decomposition of  $\mu$ 
2  $ccs \leftarrow \sigma_{\min} / \sigma$   $\triangleright$   $ccs$  helps make the execution time independent of  $\sigma$ 
3 while true do
4    $z_0 \leftarrow \text{BaseSampler}()$   $\triangleright z_0 \sim D_{\mathbb{Z}^+, \sigma_{\max}}$ 
5    $b \leftarrow \text{UniformBits}(1)$ 
6    $z \leftarrow b + (2 \cdot b - 1)z_0$   $\triangleright z \sim D_{\mathbb{Z}, \sigma_{\max}, \{0,1\}}$  (bi-modal)
7    $x \leftarrow \frac{(z-r)^2}{2\sigma^2} - \frac{z_0^2}{2\sigma_{\max}^2}$ 
8   if  $\text{BerExp}(x, ccs) = 1$  then  $\triangleright$  true with probability  $\approx ccs \cdot \exp(-x)$ 
9     return  $z + \lfloor \mu \rfloor$ 

```

Algorithm 2: BerExp(x, ccs)

Input: $x, ccs \in \mathbb{Q}^+$
Output: A single bit, equal to 1 with probability $p \approx ccs \cdot \exp(-x)$

```

1  $p \leftarrow ccs \cdot \exp(-x)$   $\triangleright \exp(x)$  implemented with polynomial approximation
2  $u \leftarrow \text{UniformBits}(64)$   $\triangleright$  Not all bits may be relevant, can be provided in chunks
3 return  $\llbracket u < \lfloor 2^{64} \cdot p \rfloor \rrbracket$   $\triangleright$  Bernoulli trial: 64-bit comparison to random bits

```

Algorithm 3: BaseSampler()

Input: -
Output: $z_0 \in \{0, \dots, 18\}$, such that approximately $z_0 \sim D_{\mathbb{Z}^+, \sigma_{\max}}$

```

1  $u \leftarrow \text{UniformBits}(72)$ 
2  $z_0 \leftarrow 0$ 
3 for  $i = 0, \dots, 17$  do  $\triangleright$  Sequentially or in parallel, but constant-time
4    $z_0 \leftarrow z_0 + \llbracket u < \text{RCDT}[i] \rrbracket$   $\triangleright$  RCDT is a table of 18 72-bit integers in [PFH+22]
5 return  $z_0$ 

```

Algorithm 4: ffSampling_n($\mathbf{t}, \mathbf{T}$)

Input: $\mathbf{t} = (t_0, t_1) \in \text{FFT}(\mathbb{Q}[x]/(x^n + 1))^2$, a FALCON tree $\mathbf{T}$
Output: $\mathbf{z} = (z_0, z_1) \in \text{FFT}(\mathbb{Z}[x]/(x^n + 1))^2$

```

1 if  $n = 1$  then
2    $\sigma \leftarrow \mathbf{T}.\text{value}$   $\triangleright$  It is always the case that  $\sigma \in [\sigma_{\min}, \sigma_{\max}]$ 
3    $z_0 \leftarrow \text{SamplerZ}(t_0, \sigma)$   $\triangleright$  Since  $n = 1$ ,  $t_i \in \mathbb{Q}$  and  $z_i \in \mathbb{Z}$ 
4    $z_1 \leftarrow \text{SamplerZ}(t_1, \sigma)$ 
5   return  $\mathbf{z} = (z_0, z_1)$ 
6  $(\ell, \mathbf{T}_0, \mathbf{T}_1) \leftarrow (\mathbf{T}.\text{value}, \mathbf{T}.\text{leftchild}, \mathbf{T}.\text{rightchild})$ 
7  $\mathbf{t}_1 \leftarrow \text{splitfft}(t_1)$   $\triangleright \mathbf{t}_0, \mathbf{t}_1 \in \text{FFT}(\mathbb{Q}[x]/(x^{n/2} + 1))^2$ 
8  $\mathbf{z}_1 \leftarrow \text{ffSampling}_{n/2}(\mathbf{t}_1, \mathbf{T}_1)$   $\triangleright$  First recursive call to  $\text{ffSampling}_{n/2}$ 
9  $z_1 \leftarrow \text{mergefft}(\mathbf{z}_1)$   $\triangleright \mathbf{z}_0, \mathbf{z}_1 \in \text{FFT}(\mathbb{Z}[x]/(x^{n/2} + 1))^2$ 
10  $t'_0 \leftarrow t_0 + (t_1 - z_1) \odot \ell$ 
11  $\mathbf{t}_0 \leftarrow \text{splitfft}(t'_0)$ 
12  $\mathbf{z}_0 \leftarrow \text{ffSampling}_{n/2}(\mathbf{t}_0, \mathbf{T}_0)$   $\triangleright$  Second recursive call to  $\text{ffSampling}_{n/2}$ 
13  $z_0 \leftarrow \text{mergefft}(\mathbf{z}_0)$ 
14 return  $\mathbf{z} = (z_0, z_1)$ 

```

so the actual latency of sampling has a direct and major impact on overall signing time, accounting for up to 72% [PFH⁺22] of execution time.

3.3 Discrete Gaussian Sampling

Obtaining a sample from $D_{\mathbb{Z},\sigma,\mu}$, where $\sigma, \mu \in \mathbb{R}$ both vary between invocations, is a difficult problem. Many traditional sampling approaches, such as table lookups, fall short. Additionally, to prevent a timing side-channel, the execution time of the algorithm must at least be independent of σ (which is part of the private key), if not constant [PRR19].

The **SamplerZ** algorithm satisfies these requirements for $\sigma \in [\sigma_{\min}, \sigma_{\max}]^3 \cap \mathbb{Q}$ and $\mu \in \mathbb{Q}$. It begins by decomposing the center μ into an integer part s and a fractional part $r \in [0, 1)$, such that $\mu = s + r$. A sample from $D_{\mathbb{Z},\sigma,r}$ (centered around r instead of μ) is obtained, and the result is shifted by s , yielding a sample from $D_{\mathbb{Z},\sigma,\mu}$ [PRR19].

To sample from $D_{\mathbb{Z},\sigma,r}$, the core sampling step first uses **BaseSampler** to sample $z_0 \sim D_{\mathbb{Z}^+, \sigma_{\max}}$, which is combined with a random bit b to obtain $z = b + (2b - 1)z_0$, where $z \sim D_{\mathbb{Z}, \sigma_{\max}, \{0,1\}}$. Here $D_{\mathbb{Z}, \sigma_{\max}, \{0,1\}}$ is a special bi-modal discrete Gaussian with modes at 0 and 1 that dominates⁴ $D_{\mathbb{Z},\sigma,r}$, so rejection sampling [RC04] can be used to obtain samples from $D_{\mathbb{Z},\sigma,r}$. This involves computing the acceptance probability $\exp(z_0^2/2\sigma_{\max}^2 - (z - \mu)^2/2\sigma^2) \cdot ccs$ and then performing a Bernoulli trial in **BerExp**. The additional term $ccs = \sigma_{\min}/\sigma$ makes the rejection probability independent of σ , which prevents a timing side-channel based on the variable number of rejections [PRR19].

3.3.1 Impact of rejection sampling in SamplerZ

The success probability of **BerExp** was empirically found to be $P_{\text{succ}} \approx 57.5752\%$ [LYN⁺24]. If the trial fails and the sample is rejected, sampling must be repeated with the same inputs but new random values (**UniformBits** outputs). Therefore, the expected number of attempts until success for a single module is ≈ 1.74 , reducing naive throughput by a factor of $\times 1.74$. If both samples in a pair were computed on parallel modules, throughput could drop by as much as $\times 2.25$. It is therefore imperative that the re-attempt latency of any practical implementation be as low as possible.

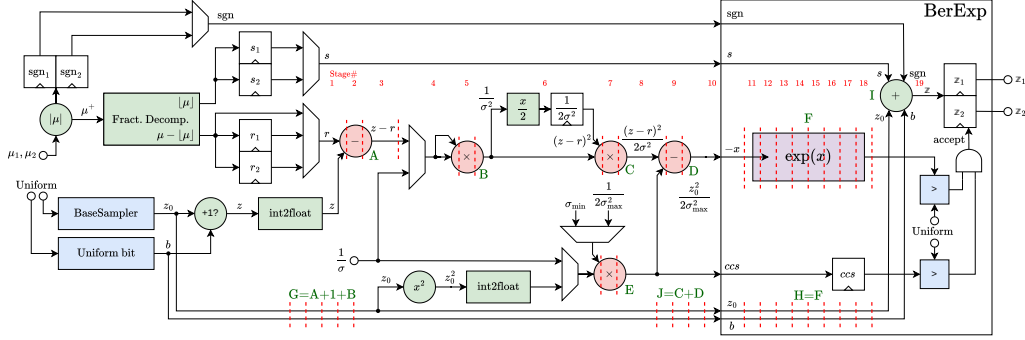
4 High-level hardware architecture

Given FP64 inputs μ_1, μ_2 , and $1/\sigma$, the accelerator produces a pair of samples $z_1 \sim D_{\mathbb{Z},\sigma,\mu_1}$ and $z_2 \sim D_{\mathbb{Z},\sigma,\mu_2}$ in just **22.2 cycles** on average, outputting z_1 and z_2 as 32-bit integers.

As shown in **Figure 1**, the accelerator is a fully pipelined design with 18 internal pipeline stages plus one output register stage. In the **first stage**, the *Boolean Knuth-Yao BaseSampler* (section 5.3) produces a random z_0 , which is combined with a random bit to compute z . In parallel, $|\mu|$ is decomposed into s and r , such that $|\mu| = s + r$ and $(s, r) \in \mathbb{N} \times [0, 1)$, by the *fractional decomposition* module (section 5.1). In the first cycle, μ_1 is presented at the μ input and decomposed; the resulting (s_1, r_1) is stored and r_1 directly forwarded to subtractor **A**. In the next cycle, μ_2 is processed in the same way. In **stages 2–10**, the design computes $x = \frac{(z-r)^2}{2\sigma^2} - \frac{z_0^2}{2\sigma_{\max}^2}$ using pipelined FP64 multipliers (**B, C, E**) and subtractors (**A, D**), instantiated here as AMD Vivado Floating-Point IPs but replaceable with other pipelined cores. In **stages 11–18**, an 8-stage FP64 exponentiation core **F** (section 5.4) computes $\exp(-x)$ for rejection sampling. In **stage 19**, the Bernoulli trial is completed by two uniform comparators (section 5.2) used in parallel as described in section 5.4.2. The first sampling attempt for μ_1 therefore completes after 19 cycles. On a successful sampling attempt, the combinational modified adder-subtractor **I** is used

³ $\sigma_{\min} = 1.2778, \sigma_{\max} = 1.8205$ [PFH⁺22]

⁴ $\mathcal{P}$ dominates $\mathcal{Q}$ if $\exists M \geq 1$ s.t. $q(x) \leq Mp(x)$ for (almost) every $x \in \text{Supp}(\mathcal{Q})$.



Components in **blue** consume uniform random bits from a central source. Components in **green** are FP and arithmetic cores implemented by hand in RTL. Components in **red** are Xilinx FP64 arithmetic cores. The component in **violet** is the FloPoCo open-source exponential operator. Select pipeline components are annotated A–J in **dark green**. The pipeline stages, separated by **red** dashed lines, are numbered also in **red** for convenience.

Figure 1: Hardware architecture of the proposed **SamplerZ** accelerator

to combine (s, sgn) and (z_0, b) , which is delayed by **G**, **J**, and **H**, and the resulting z is saved.

Because of the pipelined nature of the accelerator, after the 19 cycles required to complete the first sampling attempt, the design completes another attempt every subsequent cycle. This is a major improvement over previously used iterative rejection algorithms: each rejected sample incurs only a minimal 1 cycle penalty instead of the full **BerExp** latency.

The values $1/2\sigma^2$ and ccs , both derived from σ and therefore from private-key material, are first needed in pipeline **stages** 6 and 8 respectively. This leaves enough time to derive them using multipliers **E** and **B** before μ -derived pipeline inputs reach them. To avoid an FP64 divider in hardware, the module takes $1/\sigma$ as input instead of σ itself. This reciprocal can simply be stored instead of σ in the private key, as σ is otherwise unused, so no additional storage or online arithmetic is required.

The two values μ_1 and μ_2 , or rather r_1 and r_2 , are used alternately as inputs to the pipeline until valid samples for both z_1 and z_2 have been obtained. Thus, one attempt is performed every cycle until one of the samples succeeds. After that, an attempt for the remaining sample is completed every second cycle. Considering that 99.4 % of samples succeed within 6 attempts [LYN⁺24] and accounting for the depth of the pipeline, no latency improvement is gained by switching the input solely to the single remaining sample after the first success, as is done in [ZASM25]. By the time that change would propagate through the pipeline, the remaining sample will very likely have already succeeded anyway.

As the inputs for each new attempt are derived only from random values, the pipeline does not need to stall when random bits are unavailable. Instead, the affected attempt is simply marked as invalid and continues through the pipeline. The pipeline stages can therefore be instantiated without write-enables, saving area on ASIC and routing resources on FPGA.

The module accepts random bits from an external ChaCha20 PRNG through three independent 8-bit refill ports, which are sufficient for continuous operation. For easier deployment and testing on platforms with an integrated processing system, the included AXI4 memory-mapped register file can be used for software control of the accelerator.

5 Implementation details

5.1 Fractional decomposition of μ

At the start of **SamplerZ**, the values $\lfloor \mu \rfloor$ (the *floor*) and $\mu - \lfloor \mu \rfloor$ (the *fraction*) must be computed from the FP64 input μ . In other words, we must find $(s, r) \in \mathbb{Z} \times [0, 1)$ such that $\mu = s + r$. We refer to this operation as the *fractional decomposition* of μ .

5.1.1 Prior implementations

Xilinx FP IPs. Previous implementations [OZZ⁺25, PMKA25] used FP64-to-integer conversion IP cores from Xilinx, which internally follow IEEE754 round-to-even semantics [IEE19]. That rounding mode is undesirable for computing the *floor*, but it cannot be disabled. Instead, the floor is recovered from the rounded value using

$$\lfloor x \rfloor = \begin{cases} \lfloor x \rfloor - 1, & \text{if } x < \lfloor x \rfloor, \\ \lfloor x \rfloor, & \text{otherwise.} \end{cases} \quad (1)$$

To compute the *fraction*, the subtraction $\mu - \lfloor \mu \rfloor$ is then performed for both candidate results in parallel to minimize latency, as shown in Figure 2. This approach is inefficient: the IP cores first implement the full round-to-even behavior, after which additional logic is needed to undo it. The generic IPs also introduce enough latency that multiple instances must be used in parallel to keep the overall delay acceptable.

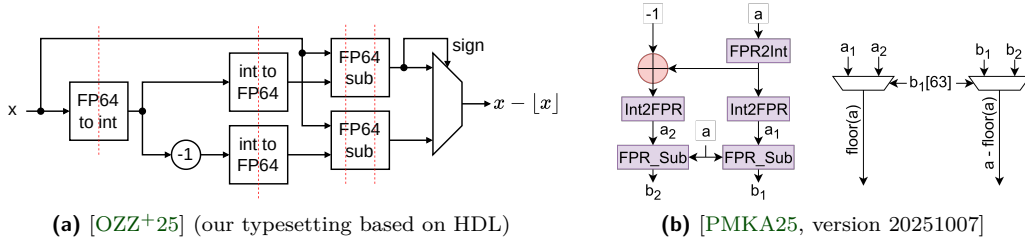


Figure 2: FP64 *floor decomposition* using IPs in prior implementations

Sequential “flt272int”. The authors in [ZASM25] use a hand-written core `flt272int`⁵, but do not provide implementation details beyond area usage in their paper. From the source code, we concluded that it is a non-pipelined 3-cycle sequential module.

5.1.2 Optimized fractional decomposition (our approach)

Instead of relying on FP64 IP cores with round-to-even semantics, we directly implemented the floor operation by hand for **non-negative** inputs by manipulating the underlying IEEE754 representation according to **DecomposeN** (Algorithm 5). If $x < 1$, no decomposition is needed and x is returned directly as the *fraction*. This also implies correct handling of subnormals, although they are not required in **SamplerZ**. Larger values of x are converted to a fixed-point representation with a barrel shifter, at which point the correct *floor* bits appear to the left of the binary radix point and can be returned without further processing. The bits to the right of the binary radix point form the *fraction*, but they must be normalized into an IEEE754-compliant FP64 number before being returned. The implementation is fully parametric and is instantiated in our **SamplerZ** for 64-bit FP and a 32-bit unsigned *floor*.

⁵<https://github.com/Shaibk/Bi-SamplerZ>

Table 1: Comparison of floating-point *fractional decomposition* implementations

Implementation	Scheduling	Frequency	Latency [†]	LUT	FF	DSP
[OZZ ⁺ 25] [*]	5-stage pipeline	235 MHz	21.3 ns	1990	712	6
[ZASM25] [*]	3-cycle sequential	870 MHz [‡]	3.5 ns	1034	314	0
DecomposeZ	combinatorial	176 MHz	5.7 ns	1359	0	0
DecomposeN	combinatorial	300 MHz	3.3 ns	669	0	0

^{*} We isolated only the *fractional decomposition* logic from the published source code and implemented it for the ZCU104 in Vivado 2025.1 for a fair comparison

[†] Total net latency if running at the maximum frequency

[‡] Maximum possible out-of-context frequency, which would be difficult to achieve in practice due to congestion and design constraints

Algorithm 5: DecomposeN(x)

Input: $x \in \mathbb{Q}^+$
Output: $(s, r) \in \mathbb{N} \times [0, 1)$, such that $x = s + r$

```

1 if  $x < 1$  then
2    $r \leftarrow x$                                 ▷ No floating-point manipulation required
3    $s \leftarrow 0$ 
4 else
5    $s.f \leftarrow \text{ToFixed}(x)$                     ▷ Recover hidden one and barrel shift left
6    $r \leftarrow \text{ToFloat}(0.f)$                   ▷ Normalize, i.e., barrel shift left
7 return  $(s, r)$ 

```

Algorithm 6: DecomposeZ(x)

Input: $x \in \mathbb{Q}$
Output: $(s, r) \in \mathbb{Z} \times [0, 1)$, such that $x = s + r$

```

1  $s, r' \leftarrow \text{DecomposeN}(|x|)$ 
2 if  $x < 0$  then
3   if  $r' \neq 0$  then                                ▷  $r$  would be negative, adjust
4      $\text{return } (-s - 1, 1 - r')$                   ▷ Performed on  $f$  while it is in fixed-point
5   else
6      $\text{return } (-s, 0)$                                 ▷ Integer, adjust only  $s$ 
7 else
8    $\text{return } (s, r')$                                 ▷ Positive, no adjustment needed

```

Algorithm 7: SamplerZ_wrapper(μ, σ)

Input: $\mu \in \mathbb{Q}, \sigma \in [\sigma_{\min}, \sigma_{\max}]$
Output: $z \sim D_{\mathbb{Z}, \sigma, \mu}$

```

1  $z' \leftarrow \text{SamplerZ}(|\mu|, \sigma)$                 ▷ Can use DecomposeN because  $|\mu| \in \mathbb{Q}^+$ 
2 if  $\mu < 0$  then
3    $\text{return } -z'$ 
4 else
5    $\text{return } z'$ 

```

Decomposition of negative numbers. The above algorithm does not work for $x < 0$, which makes it unsuitable for direct use in **SamplerZ**, whose inputs may be negative. Support for signed inputs can be added according to **DecomposeZ** (Algorithm 6) at the cost of two additional subtractors (lines 4 and 6), whereas **DecomposeN** requires none. This nearly doubles both the critical-path length and the area (compare **DecomposeZ** and **DecomposeN** in Table 1), so we instead avoid negative μ inside **SamplerZ** altogether.

Ensuring non-negative μ inside SamplerZ. There is a strong incentive in both latency and area to avoid negative μ inside **SamplerZ**. Fortunately, by the symmetry of the discrete Gaussian, we can replace a negative center with its absolute value and correct the sign afterward.

The **SamplerZ** algorithm itself does not directly sample from $D_{\mathbb{Z},\sigma,\mu}$, but uses a sample from $D_{\mathbb{Z},\sigma,r}$ to compute the final sample by *translating* the Gaussian:

$$D_{\mathbb{Z},\sigma,\mu} = \lfloor \mu \rfloor + D_{\mathbb{Z},\sigma,\mu - \lfloor \mu \rfloor} = \lfloor \mu \rfloor + D_{\mathbb{Z},\sigma,r} \quad (2)$$

For discrete Gaussians, a similar identity for *negation* can also be derived⁶:

$$D_{\mathbb{Z},\sigma,\mu} = -D_{\mathbb{Z},\sigma,-\mu} = \text{sgn}_*(\mu) \cdot D_{\mathbb{Z},\sigma,|\mu|} \quad (3)$$

Using this identity, **SamplerZ_wrapper** wraps **SamplerZ** so that the internal center is always non-negative. This in turn allows **DecomposeN** to be used inside **SamplerZ** and saves resources. Note that the final conditional inversion does not incur any additional LUT or latency cost: as the output of **SamplerZ** is a sum, we simply use a modified adder-subtractor to merge the inversion into the addition.

5.1.3 Implementation results and comparison

In Table 1, we compare isolated implementation results of our *fractional decomposition* cores **DecomposeN** and **DecomposeZ** with the methods used in [OZZ⁺25] and [ZASM25]. For both prior implementations, source code is available, but *fractional decomposition* was not provided as an isolated module. We therefore extracted only the relevant logic and implemented it for the ZCU104 board in Vivado 2025.1 at the highest achievable frequency for a fair comparison. The implementation in [PMKA25] is based on the same principle as [OZZ⁺25] (Figure 2) and was not specifically evaluated, as its source code is not available.

Our core **DecomposeN** stands out by not requiring any pipelining at 300 MHz, which is higher than the $F_{\max}$ of all previous FPGA **SamplerZ** implementations [KA23, YSZ⁺24, ZASM25, OZZ⁺25, LYN⁺24, SAW⁺23], while requiring $1.5\times$ to $3\times$ fewer LUTs and no DSPs. While [ZASM25]’s module can achieve a very high $F_{\max}$, that frequency is difficult to exploit in practice, and its sequential nature introduces extra complexity.

5.2 Optimized uniform comparison

The final operation of **BerExp** is to compare a computed 64-bit value to a 64-bit uniformly distributed random value. The specification [PFH⁺22] suggests performing this comparison in 8-bit chunks starting with the most significant chunk and stopping once the comparison is decided. This reduces randomness consumption in a software-friendly way, as there is only a $1/256$ chance of needing more than one byte of randomness. The expected average number of 8-bit comparisons is 1.003925, so the comparison consumes only 8.0314 bits per invocation on average.

⁶*Proof.* $\rho_{\sigma,\mu}(x) = \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) = \rho_{\sigma,-\mu}(-x) = \rho_{\sigma,|\mu|}(\text{sgn}_*(\mu) \cdot x)$.

5.2.1 Prior implementations

In software and in previous implementations [OZZ⁺25, ZASM25, KA23, YSZ⁺24], these one-byte comparisons are performed sequentially in a loop. In the FP-Exp variant of [PMKA25], this loop is fully unrolled, slightly increasing area while improving the average latency by 0.39% and ensuring predictable single-cycle completion.

5.2.2 Our approach to randomness consumption optimization

In hardware, we can push this idea even further by performing the comparison sequentially bit-by-bit. In the ideal case, that would reduce average consumption to 2 bits per comparison, but it would also increase the latency to 2 cycles on average, with a worst case of 64 cycles. At the other extreme, performing all comparisons in parallel would preserve the low randomness consumption with single-cycle latency, but refilling the component with a variable amount of 1–64 bits every cycle is complex and would incur a large area overhead.

For our implementation, we therefore adopt a compromise: a small number of most significant bits (*hot bits*) are compared bit-by-bit, while the remaining bits are compared in *groups* of several bits. This retains most of the randomness savings while reducing resource usage. It also enables an efficient refill strategy, shown in Figure 3: the *hot bits* are refilled with shift registers acting as parallel-to-serial converters, while the *groups* are refilled directly. Bit groups have one backup buffer and hot bits have one or more; these help avoid starvation when randomness cannot be refilled from the source in time for the current cycle. Assuming that only one *group* or *hot bit* needs a refill in any given cycle, the module can use a simple blocking randomness input with the width of a group (8 bits in our case), similar to the byte-by-byte interface from the specification. To save resources, most *bit groups* are implemented as a simple shift-register array instead of a true FIFO. This means that when a value is consumed from the same *bit group* twice in a row without refill, a “bubble” is introduced, which causes a failed sample upon reaching the consumer. This is unlikely for insignificant *bit groups*, but occurs quite often for a few of the most significant ones, so those are equipped with a FIFO-style forwarding option that provides additional refill leniency and makes bubbles highly improbable.

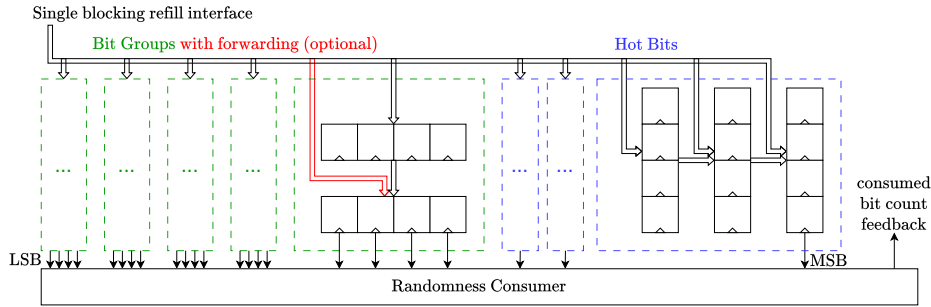


Figure 3: Optimized randomness refill logic for uniform comparison

Parameters and implementation results. Our refill buffer module is highly parametric, which allowed us to experimentally select the best parameters. Figure 4a shows the effect of *group* size and *hot bit* count on average randomness consumption per comparison. For compatibility with existing designs, we chose a refill width (and thus group size) of 8 bits. We settled on 5 *hot bits* for our comparator, as the benefit of adding more quickly diminishes. This yields an average consumption of **2.1885** bits per 64-bit comparison instead of the optimal 2 bits, which is still a **3.67×** improvement over byte-by-byte

comparison. The number of *bit groups* equipped with the fast-refill path, and the depth of backup buffers for *hot bits*, must be chosen so that “bubbles” occur rarely, as they cause invalid samples and therefore lower the overall sampling rate. We found 1 fast-refill group and 1 hot-bit backup buffer to be sufficient for uniform comparison. In this configuration, the refillable comparator consumes 415 LUTs and 284 FFs.

Application to BaseSampler. This refill buffer can also be applied to the Boolean **BaseSampler** described in section 5.3, which consumes random bits in a similar way to uniform comparison. The effect of *hot bits* on **BaseSampler** refill is shown in Figure 4b.⁷ While the theoretical lowest randomness consumption of the Boolean **BaseSampler** is 3.24 bits/sample, a practical configuration with 6 *hot bits*, 2 fast-refill groups, and 2 *hot-bit* backup buffers consumes **3.52 bits/sample**. This is still a **20.45×** reduction compared with the 72 bits/sample required by previous implementations, at only a small area premium (see Table 2).

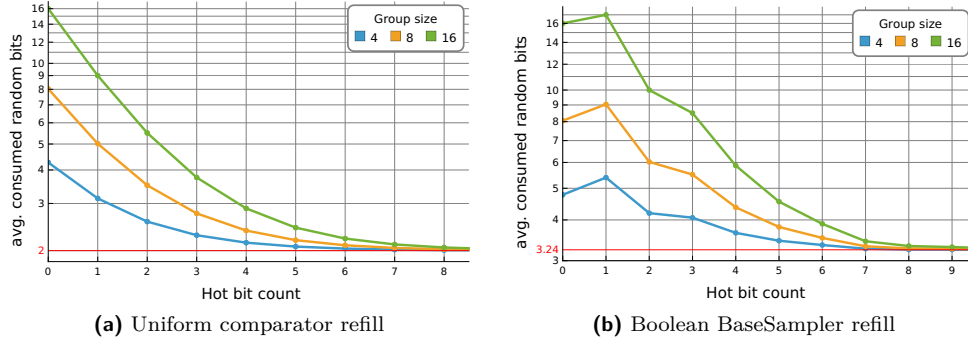


Figure 4: Effect of *hot bits* and *group size* on randomness consumption. Theoretical minimum (comparison bit-by-bit) in red.

5.3 Boolean minimization BaseSampler

The **BaseSampler** algorithm ensures a random $z_0 \in \{0, \dots, 18\}$ from a distribution close to the half-Gaussian $D_{\mathbb{Z}^+, \sigma_{\max}}$. It works by comparing a uniformly distributed 72-bit random value with a Reverse Cumulative Distribution Table (RCDT), requiring 18 wide integer comparisons and then counting the number of successes. In software, these are performed sequentially and always to completion to ensure constant-time execution.

5.3.1 Prior implementations

In hardware, the specification algorithm can be implemented in various ways, which differ mainly in comparison parallelism and counting method. [YSZ⁺24] instantiates a single comparator and performs the comparisons sequentially while incrementing an accumulator. [OZZ⁺25] uses two sets of 9 constant comparators, with the results summed across two clock cycles to break up the summation critical path. [ZASM25] instantiates 18 parallel constant comparators and proposes logic based on tri-state buffers in ASIC, and a priority encoder on FPGA, to avoid the long summation.

⁷The plot shows that a single *hot bit* performs worse than none. This is due to the structure of **BaseSampler**’s PDT, as it always consumes at least two bits: with a single *hot bit*, the core is forced to always refill both a *group* and the *hot bit*, instead of just the *group*.

5.3.2 Our optimized BaseSampler

We use the software technique from [KRVV19], a variant of Knuth-Yao sampling [KY76], which was adapted to hardware by [LG20]. The target probability distribution table (PDT) is transformed into a discrete distribution generation (DDG) binary tree (see [LG20, Fig. 1]). A random walk through a correctly constructed DDG tree, which ends at a leaf with a specific sample value, produces that sample with exactly the probability specified in the PDT. In [KRVV19], the DDG tree was then transformed into a truth table (Figure 5), where the input is the string of all decisions (bits) taken while walking the DDG. If the tree is as unbalanced (“left-packed”) as possible, the truth table is highly structured, with long prefixes of *don’t cares* and long suffixes of *1s*, allowing efficient implementation by heuristic optimizers. [KRVV19] processed the truth table using Espresso [BHMS84] to obtain Boolean algebra expressions, which could be evaluated *in software*, yielding a slow but *constant-time* sampler. [LG20] also used Espresso to produce minimized expressions, which were described using VHDL and then synthesized with Vivado.

We updated⁸ the code from [Kar19] and used it to generate the truth table for **BaseSampler** in Espresso format, which we then post-processed into a SystemVerilog module containing a single `casez` statement, and used Vivado’s built-in optimizer for synthesis, which provided satisfactory results. We call this module the *Boolean BaseSampler* (`boole` in Table 2).

To optimize this module further, we applied Espresso single-output minimization to the base table, followed by a long run of ABC’s [BM10] `deepsyn` [MMW⁺25]. Notably, single-output minimization seems to perform better on the **BaseSampler** function than multi-output, and the multi-level synthesis of ABC is more effective than Espresso’s two-level minimization. The resulting logic network was converted to VHDL and synthesized with Vivado.⁹

The module has been verified semantically by reconstructing the PDT from HDL and statistically through simulation, confirming that its output distribution matches that of FALCON’s own **BaseSampler**. Thus, the module can safely be used as a drop-in replacement for the comparator-based version.

Inputs	Outputs	
Random bits	Sample	Consumed random bits
-----00	00001	2
-----10	00000	2
-----001	00010	3
-----0101	00011	4
...	...	...
-----1100111111111111	00001	18
-----0100011111111111	01001	19
-----1010011111111111	01010	20
-----1110111111111111	01011	20
-----1001011111111111	01111	21
...	...	...
-111011111111111111111111111111	00110	31
-101011111111111111111111111111	01000	31
011111111111111111111111111111	00011	32
111111111111111111111111111111	00010	32

Figure 5: Example Boolean sampler truth table with “used bit count reporting”

⁸Having been published in 2019, [KRVV19] works with an outdated FALCON version, which has a different **BaseSampler** PDT than current FALCON.

⁹We thank doc. Ing. Petr Fišer, Ph.D. for running the synthesis and providing insight into this function.

Reducing randomness consumption. In the original **BaseSampler**, every bit of the **UniformBits(72)** input has a high chance of being relevant. In the *Boolean BaseSampler*, the inputs, starting with the rightmost bit, correspond to the decisions taken while walking the DDG tree. Once a leaf is reached, which is very likely in a highly unbalanced tree, the remaining input bits are irrelevant to the outcome and are therefore marked as *don’t care* in the truth table. It follows that only a contiguous substring starting with the rightmost bit is actually needed to determine the outcome, and that substring is often short.

We can annotate each row of the module’s truth table with the number of rightmost bits (decisions) that were actually relevant to the walk represented by that row, namely the count of non-*don’t-care* bits (see Figure 5). We then extend the truth table to emit this count as a second output, at the cost of 40% more LUTs (**bool_smart** in Table 2).

A full 72-bit random value must be available to the module in the unlikely (on the order of 2^{-72}) event that all 72 bits are relevant. After each operation, however, we replenish only the bits that were actually consumed, based on the module’s second output. From the truth table, we calculated that, if optimally refilled, the module would need only **3.24** new random bits for each sample on average, which is **22×** less than the 72 bits consumed by the original **BaseSampler**.

Implementation results and comparison. Table 2 presents FPGA implementation results for several **BaseSampler** variants. The comparator-based implementations from [PMKA25, ZASM25, OZZ⁺25] described in section 5.3.1 occupy very similar post-implementation area and all meet timing comfortably above the operating point relevant to our design. Their main differences therefore lie in structure and input interface rather than in maximum achievable clock frequency. [LG20] offers an 11 % improvement over the comparator variant.

The **bool** variant proposed in this work is at least **2.3×** more compact than the comparator-based designs when synthesized using Vivado directly, and high-effort optimization using ABC achieves **3×**. The **bool_smart** variant adds input-bit usage reporting, allowing the randomness consumption to drop by up to **22×** at the cost of 1.4× more LUTs than **bool** (same Vivado optimizer), plus external refill logic. Our practical **bool_refill** variant integrates the refillable uniform buffer from section 5.2, trading some wasted randomness and additional FFs for a simple 8-bit blocking uniform input. It consumes **3.52** random bits per sample on average, a **20.5×** reduction over prior **BaseSampler** implementations, at the cost of 260 additional buffer FFs and 30 additional LUTs.

5.4 Pipelined exponentiation in BerExp

The **BerExp** subroutine computes $p = \exp(-x) \cdot ccs$ with high precision, then $\lfloor 2^{64} \cdot p \rfloor$ and compares the resulting integer to 64 bits of uniform randomness. In the FALCON specification, $\exp(x)$ is computed by evaluating a degree-12 polynomial approximation in 64-bit integers. This approach works well in software, where fast multipliers are readily available, but is either expensive or slow in hardware. However, because the calculation is already carried out in integers, the $\lfloor 2^{64} \cdot p \rfloor$ operation is effectively free.

5.4.1 Prior implementations

Most previous implementations [OZZ⁺25, ZASM25, KA23, YSZ⁺24] implemented $\exp(x)$ by evaluating the polynomial sequentially on a single 64-bit integer multiplier, consuming moderate resources but incurring a large latency (40–47 cycles [OZZ⁺25] and 34–41 cycles [ZASM25] for full **BerExp**). Furthermore, because the calculation has to be repeated on failure, this sequential module becomes the bottleneck even when the high-level design is partially pipelined, as it has the highest initiation interval among the pipeline components.

The authors in [PMKA25] evaluated two additional approaches to exponentiation. First, they evaluated Estrin’s form of the same polynomial (suggested in [PRR19]), which

Table 2: Comparison of **BaseSampler** implementations

Variant	Main Optimizer	Area		Cycles/ sample	Uniform input interface		
		LUT	FF [†]		Width	Blocking	Bits/sample
[OZZ ⁺ 25]*	Vivado	666	77	2	72	Yes	72
[ZASM25]*	Vivado	663	72	1	72	No	72
[PMKA25]*	Vivado	655	72	1	72	No	72
[LG20]	Espresso	590	-	-	-	-	-
boole [♦]	Vivado	277	72	1	72	No	72
boole [♦]	ABC [#]	217	72	1	72	No	72
boole_smart [♦]	Vivado	395	72	1	2-72 [‡]	No	avg. 3.24 [‡]
boole_refill [♦]	Vivado	690	332	1	8	Yes	avg. 3.52

[♦] This work

* Synthesized and implemented based on published source code

[†] Includes buffer for uniform input

[‡] Requires precise external refill of only used bits

[#] **espresso** -Dso → ABC **deepsyn** → VHDL → Vivado

enables some multiplication parallelism, reducing latency but increasing area. Second, they instantiated a pipelined FP64 exponential operator from Xilinx, reducing total **BerExp** latency to just 12 cycles. This approach is similar to ours and was developed independently¹⁰, but they do not fully exploit the pipelined nature of the operator, leaving performance on the table. Additionally, working in the FP domain introduces complexity: multiplication by 2^{64} and conversion to integers (for the uniform comparison) are no longer free. Based on reported resource-usage numbers and diagrams, it appears that the authors instantiate two Xilinx FP64 multiplier IPs and one FP64-integer converter IP to finalize the computation, though this is not explicitly documented.

5.4.2 Our approach

Identifying rejection overhead as a major problem in section 3.3.1, we set out to fully pipeline **BerExp** instead of relying on sequential datapaths. We considered the Xilinx FP64 exponential IP used in [PMKA25], but ultimately chose the open-source FloPoCo IP-generator framework [dD19] and instantiated its pipelined FP64 exponential core [dDP10]. Table 3 compares the polynomial approximation method from [OZZ⁺25], the Xilinx-IP approach from [PMKA25], and our FloPoCo-based design. Relative to the Xilinx core, the FloPoCo core offers the following advantages:

- The core’s arithmetic algorithm is well documented and non-proprietary [dDP10].
- The generated core is unencrypted VHDL licensed under the LGPL, so it is not vendor-locked to Xilinx devices.
- The core can be generated for floating-point formats of arbitrary widths, not just FP32 and FP64, and has configurable precision [dDP10]. When modifying **BerExp**’s exponential algorithm, it is critical to understand the core’s precision and verify that it meets the requirements in [PRR19], where a relative error bound of $\epsilon \leq 2^{-43.5}$ for the $\exp(x)$ submodule was derived. Our instantiation of the core satisfies $\epsilon \leq 2^{-52}$ (1 FP64 ULP), which is both by design [dDP10] and was empirically verified by fuzzing in simulation.

¹⁰[PMKA25] was first posted to IACR in August 2025, while our approach was first publicly released in June 2025 [Voj25].

- The generator supports automatic pipelining based on FPGA primitive timing estimates and can be extended with custom architectures.
- Fewer DSPs (**4.33×**) and BRAMs (**1.67×**), which are typically at a premium in FPGAs, although at the cost of $3.54\times$ more LUTs and $2.5\times$ more FFs (Table 3).
- **1.51×** higher maximum frequency and lower latency (Table 3), which allows higher throughput when the accelerator fully takes advantage of pipelining.

With this IP core, **BerExp** can be implemented as a full pipeline with a latency of 11 cycles, as shown in Figure 6a, and we can reduce that latency even further.

Parallel comparison of *ccs*. Instead of multiplying probabilities in **BerExp**, we can perform two independent Bernoulli trials in parallel. For $X, Y \sim \text{Uniform}(0, 1)$ i.i.d.:

$$\mathbb{P}[X < ccs \cdot e^{-x}] = \mathbb{P}[X < ccs] \cdot \mathbb{P}[Y < e^{-x}] = \mathbb{P}[(X < ccs) \cap (Y < e^{-x})] \quad (4)$$

This avoids the multiplication and therefore reduces latency, at the cost of doubling the number of uniform comparisons. Because section 5.2 reduces the randomness cost of one comparison to 2.1885 bits, this trade-off remains attractive. Figure 6b shows the resulting pipelined **BerExp** implementation, which has a latency of just **8 cycles** and consumes at most **4.377 bits** of randomness per operation. In our implementation, if the *ccs*-branch comparison fails, the second comparison is skipped and its randomness is not consumed, which further reduces the effective average randomness usage.

Table 3: Comparison of FP64 exponential implementations

Variant	Area				Performance			Latency ratio
	LUT	FF	DSP	BRAM	Cycles	F _{max}	Latency	
Poly. [OZZ ⁺ 25]*	2904	1237	58	0	39	185	210.8 ns	$5.86\times$
Xilinx [PMKA25]	693	575	26	2.5	8	147.1	54.4 ns	$1.51\times$
FloPoCo [♦]	2452	1436	6	1.5	8 [†]	222.2	36.0 ns	$1.00\times$

[♦] This work

* Synthesized and implemented based on modified published source code—only the FP64 exponentiation was isolated from the BerExp implementation.

[†] Including extra output register stage for fair comparison.

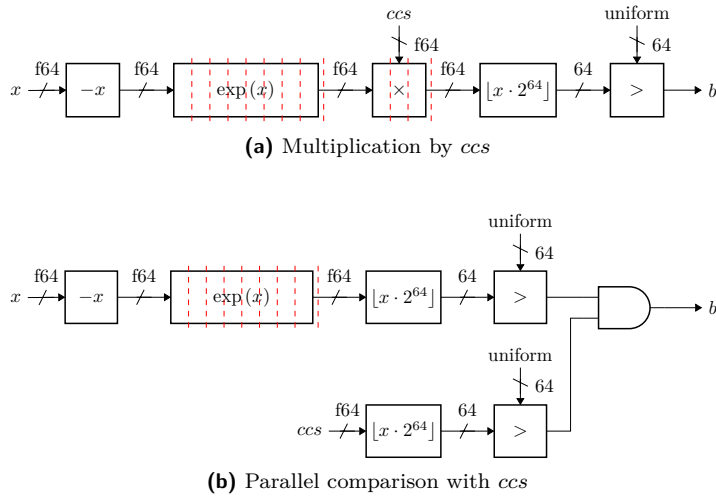


Figure 6: Pipelined **BerExp** implementation variants using the FloPoCo $\exp(x)$ operator

Scaling and conversion to integers. Given that we work with FP64 numbers, the operation $\lfloor 2^{64} \cdot p \rfloor$ is slightly more involved than in the integer domain. Unlike [PMKA25], which instantiates Xilinx FP64 multiplication and rounding IPs for this task, we observe that scaling an IEEE754 number by 2^{64} requires only exponent manipulation, not an actual multiplication. Because we also do not need to support subnormals or special FP values in this block, the subsequent conversion to integer is simple enough to implement directly. We therefore combine both steps in hand-written HDL, allowing the logic to synthesize into a small combinational circuit that requires no DSPs and adds no pipeline stage.

Implementation results. In Table 4, we compare complete **BerExp** implementations. Although [PMKA25] already improves on [OZZ⁺25, ZASM25] by adopting a pipelined exponential core, our implementation still lowers latency by **2.47×**. Relative to their C1 design, ours uses 2477 more LUTs and 852 more FFs, but saves 36 DSPs and 36 KiB of BRAM. That DSP and BRAM reduction is particularly valuable on resource-constrained devices. In addition, about 400 LUTs and 400 FFs in our result come from the refill logic (section 5.2) that keeps the randomness cost below 4.377 bits per **BerExp** invocation (Table 7); in environments where random bits are plentiful, that logic could be removed to save area.

Table 4: Comparison of **BerExp** implementations

Variant	Area				Performance		Latency ratio	Rand. usage [‡]
	LUT	FF	DSP	BRAM	Cycles	Latency [†]		
[OZZ ⁺ 25]*	3095	1301	58	0	40–47	216.4 ns	6.01×	8.031
[ZASM25] [#]	1343	266	32	0	34–41	226.9 ns	6.30×	8.031
[PMKA25] [¶]	1272	1216	42	2.5	12	88.9 ns	2.47×	8–64 [§]
This work	3749	2068	6	1.5	8	36.0 ns	1.00×	≤ 4.377

[†] Average latency at actual SamplerZ design frequency.

[‡] Average number of uniform random bits consumed per operation.

* Synthesized and implemented based on published source code.

[#] We consider `For_loop`, `CMP` and `SUB63` together as **BerExp**.

[¶] Version C1 (best area and frequency)

[§] Authors note that the uniform comparison loop is fully unrolled to reduce latency, but do not specify if only used bytes are consumed and refilled, which is required to achieve 8.031 bits.

6 Comparing SamplerZ implementation performance

To compare **SamplerZ** implementations fairly, we propose Rejection-Inclusive Pair Sampling Time (RIPST) as a replacement for [PMKA25]’s Rejection-Adjusted Execution Time (RAET) metric, which assumes two parallel hardware modules are instantiated to obtain the two samples in one pair. RIPST instead measures the implementation’s actual average time to produce a pair of accepted samples *on a single instance*. This definition is more suitable for comparisons across **SamplerZ** architectures (including designs such as ours that obtain both samples efficiently on a single instance) and better describes real-world use of the accelerator. RIPST may be given either in cycles (subject to frequency differences), or directly in ns for a given platform.

For designs that cannot natively obtain two independent samples on one instance, we use a serial model in which the second sampling starts as soon as the first sample is accepted. Although this increases raw latency relative to a theoretical two-module setup,

Table 5: Timing analysis of **SamplerZ** implementations in cycles. RIPST represents the average number of cycles required to produce two accepted samples on a single module instance.

Variant	P_s	T_f	T_n	RIPST	Calculation
[KA23]	57.6%	1807	1807 [†]	6277.0	$E[T_{\text{Serial}}]$
[YSZ ⁺ 24]	57.6%	104	104 [†]	361.3	$E[T_{\text{Serial}}]$
[OZZ ⁺ 25]	57.6%	67 [*]	40 [*]	192.9	$E[T_{\text{Serial}}]$
[ZASM25]	57.6%	59	34 [*]	106.1	[ZASM25]
[PMKA25] [§] C1	57.6%	22	22 [#]	76.4	$E[T_{\text{Serial}}]$
[PMKA25] [§] C3	92.4% [‡]	22	22 [#]	47.6	$E[T_{\text{Serial}}]$
This work	57.6%	19	1	22.2	$E[T_{\text{Alternating}}]$

[†] Presumed to be the same as T_f .

[‡] Uses 3 parallel sampling loops for each attempt, increasing P_s .

^{*} Minimum possible value.

[#] Not reported, but verified by reproducing their own RAET calculations.

[§] Non-peer-reviewed IACR Cryptology ePrint Archive publication, included as SoA reference.

it gives a better time-area product: because of rejections, when two parallel modules run to completion, the faster one must wait for the slower one, resulting in lower throughput and therefore a higher time-area product than a single module running two samplings sequentially. Instantiating multiple parallel compute units is always possible, but generally results in a worse time-area product. In practice, instantiating two modules may also be undesirable due to resource constraints.

Additionally, we use the RIPST-area product (RIPST-AP), the product of RIPST (ns) and area, analogously to the conventional time-area product.

6.1 RIPST calculation

We estimate the RIPST of previous implementations from the values T_f (time until the first sampling attempt completes), T_n (time for each subsequent attempt), and P_s (attempt success probability), which we collected from their publications, and present the results in Table 5. We define $P_f = 1 - P_s$ in this section for brevity. The success probability of **SamplerZ** is $P_s \approx 57.5752\%$ [LYN⁺24], but in some variants of [PMKA25], multiple parallel instances of the sampling loop can increase this probability to $1 - P_f^n$ for n sampling loops. The expected time to obtain two samples in series on a single accelerator with these characteristics is $E[T_{\text{Serial}}]$ in eq. (5), and the runtime of our alternating sampling pipeline is $E[T_{\text{Alternating}}]$ in eq. (6). The equations are intentionally left unsimplified to preserve their meaning.

$$E[T_{\text{Serial}}] = 2 \left(P_s T_f + P_f \left(T_f + \frac{T_n}{P_s} \right) \right) \quad (5)$$

$$E[T_{\text{Alternating}}] = P_s \left(T_f + \left(\frac{2}{P_s} - 1 \right) T_n \right) + P_f \left(T_f + \frac{T_n}{P_s} + \left(\frac{2}{P_s} - 1 \right) T_n \right) \quad (6)$$

If one sets $T_f = T_n = 1$ *attempt* instead of substituting cycle or latency figures, the equations can be used to calculate the expected number of sampling *attempts* required

to obtain both samples. For instance, $E[T_{\text{Alternating}}] \approx 4.21058$ attempts, which can also be obtained more intuitively by describing the alternating pipeline operation as $1/P_s + (2/P_s - 1)$.

For reference, although this is irrelevant to our implementation and comparison, we also include formulas for the time to obtain two samples on *two parallel modules*. If the first-to-succeed module stores its result and waits for the other module, the runtime is $E[T_{\text{ParLast}}]$ in eq. (7). If, however, both modules run indiscriminately (throwing away successful results) until *both* modules succeed *at the same time*, they take $E[T_{\text{ParBoth}}]$ in eq. (8). Although this is not documented, we were able to verify that [PMKA25] uses $E[T_{\text{ParBoth}}]$ for their RAET calculations, whereas an optimized implementation would likely behave according to $E[T_{\text{ParLast}}]$. This distinction further motivates RIPST as a replacement for [PMKA25]’s RAET. Additionally, this should be taken into account if citing RAET numbers from [PMKA25], especially since [PMKA25] is a non-peer-reviewed IACR ePrint Archive publication.

$$E[T_{\text{ParLast}}] = P_s^2 T_f + 2P_s P_f \left(T_f + \frac{T_n}{P_s} \right) + P_f^2 \left(T_f + \left(\frac{2}{P_s} - \frac{1}{1 - P_f^2} \right) T_n \right) \quad (7)$$

$$E[T_{\text{ParBoth}}] = P_s^2 T_f + (1 - P_s^2) \left(T_f + \frac{T_n}{P_s^2} \right) \quad (8)$$

7 Implementation results

Table 6 compares our design with prior **SamplerZ** implementations in terms of post-route FPGA area on the ZCU104 and latency, including RIPST. Area and performance are taken from the respective publications, while RIPST is calculated based on frequency and Table 5. RIPST-AP is calculated as the product of RIPST (ns) and area, and normalized against this work.

Our design achieves the lowest first-attempt latency reported on FPGA so far, at **19 cycles** (18 cycles without the output register), while also reaching the highest operating frequency among the compared FPGA implementations, namely **222 MHz**. Combining the one-cycle reattempt penalty of the fully pipelined datapath with the alternating-sampling scheme reduces the average rejection overhead to just **3.2 cycles**, yielding the lowest RIPST of **22.2 cycles** or **100 ns** to date. Compared with recent low-latency designs [OZZ⁺25, ZASM25, PMKA25], our implementation is also smaller in every major resource except BRAM and reduces DSP usage by **3.1×–8.4×**, making it practical even on devices with limited DSP budgets. Relative to [PMKA25] C1, which features the best prior RIPST-AP and is also the closest prior design to ours in overall area, we reduce RIPST by **5.66×** while using roughly 600 fewer FFs, **64 fewer DSPs**, and one fewer BRAM36. Overall, the resulting RIPST-AP improvement ranges from **5.6×** to **30.8×**, depending on the compared design and resource class.

A detailed per-module area and latency breakdown after place-and-route on the ZCU104 in Vivado 2025.1 is given in Table 7. As in previous works [KA23, PMKA25], we exclude the PRNG, AXI interface, and sample-histogram logic from the main comparison because they are not the focus of this work and were not specifically optimized. We therefore evaluate the **samplerz** module itself as our contribution to **SamplerZ** acceleration.

Table 6: Comparison of **SamplerZ** implementations for obtaining two samples on a single module

Variant	Area	Performance		RIPST		RIPST-AP (norm.)
	LUT/FF/DSP/BRAM	Latency (cycles)	$F_{\max}$ (MHz)	(cycles)	(ns)	LUT/FF/DSP/BRAM
[KA23] ^{*†}	4068/3202/9/2	1807	121	6.3k	51.8k	300/489/156/691
[YSZ ⁺ 24] [‡]	4761/692/9/0	104	83	361.3	4352.6	29.4/8.9/13.1/-
[OZZ ⁺ 25] ^{†‡}	9368/4527/76/0	67	185	192.9	1043.0	13.9/13.9/26.4/-
[ZASM25] [†]	7603/3880/85/0	59	150	106.1	707.2	7.6/8.1/20.0/-
[PMKA25] [§] C1 [‡]	7k/4k/94/2.5	22	135	76.4	566.1	5.6/6.7/17.7/9.4
[PMKA25] [§] C3 [‡]	15k/9k/254/7.5	22	131	47.6	363.6	7.7/9.6/30.8/18.2
This work	7039/3431/30/1.5	19	222	22.2	100.0	1.0/1.0/1.0/1.0

[†] Utilization of the ChaCha20 module is subtracted for a fair comparison.

[‡] Two samples are computed serially; this gives a better RIPST-AP than two parallel modules.

^{*} Implemented on the Zedboard XC7Z020 platform instead of the ZCU104. AXI communication area and latency overhead are not excluded because part of **SamplerZ** is executed in software, so the hardware cannot be treated as a standalone module.

[§] Non-peer-reviewed IACR Cryptology ePrint Archive publication, included as SoA reference.

Table 7: Hierarchical implementation results of `samplerz_axi_top`

Module	LUT	FF	DSP	BRAM	Latency
<code>samplerz_axi_top</code>	15283	11572	30	1.5	N/A
└─ <code>axi</code>	4515	874	–	–	N/A
└─ <code>histogram (2×)</code>	874	2594	–	–	N/A
└─ <code>prng_refill</code>	1931	2137	–	–	N/A
└─ <code>chacha20</code>	1805	1125	–	–	N/A
└─ <code>samplerz</code>	7039	3431	30	1.5	18
└─ <code>fp_int_frac</code>	664	–	–	–	–
└─ <code>refillable_base_sampler</code>	690	332	–	–	–
└─ <code>refill_uniform_buf</code>	273	332	–	–	–
└─ <code>basesampler_pla_smart</code>	401	–	–	–	–
└─ <code>fp_sub (2×)</code>	718	238	3	–	2
└─ <code>fp_mul (3×)</code>	174	83	6	–	2
└─ <code>fp_div2n</code>	67	–	–	–	–
└─ <code>i2f_small_z</code>	6	–	–	–	–
└─ <code>i2f_small_z0_sq</code>	28	–	–	–	–
└─ <code>berexp</code>	3749	2068	6	1.5	8
└─ <code>fp_exp</code>	2452	1436	6	1.5	7
└─ <code>float_2exp_floor</code>	243	–	–	–	–
└─ <code>refillable_uniform_cmp (2×)</code>	415	284	–	–	–
└─ <code>refill_uniform_buf</code>	213	284	–	–	–
└─ <code>lazy_uniform_cmp</code>	192	–	–	–	–
└─ <code>mu_z_adder</code>	32	–	–	–	–

For each module, post-implementation results including submodules are listed.

For modules with multiple instances, area of a single module is listed.

8 Conclusion

In this paper, we introduced the first fully pipelined architecture for FALCON’s Gaussian sampler, **SamplerZ**, and showed that rejection sampling need not remain the dominant bottleneck in hardware signing. Under our Rejection-Inclusive Pair Sampling Time (RIPST) evaluation framework, the design improves on the state of the art in both speed and resource usage. Relative to the strongest prior RIPST-area-product baseline, it lowers RIPST by **5.6×** while also reducing area, most notably by using **3.1×** fewer DSPs. Across all compared implementations, the resulting RIPST-area product improves by at least **5.6×**.

Beyond raw performance, our hardware-software co-design also yields substantial gains in randomness efficiency through the optimized **BaseSampler** and uniform-comparison logic. The design is open-source, implemented in SystemVerilog, and depends on only two proprietary Xilinx IP cores, both isolated behind simple wrappers and therefore straightforward to replace with open alternatives.

These results demonstrate that FALCON’s Gaussian sampling can be implemented efficiently in hardware and that a carefully pipelined design removes a major obstacle to high-performance post-quantum signature generation.

9 Acknowledgments

This work was supported by the Student Summer Research Program 2025 of FIT CTU in Prague and was carried out within The Advanced Chip Design and Research Center (ACDRC) project focused on innovation and collaboration in semiconductor research and education between Taiwan and the Czech Republic. This work was also supported by the Czech Technical University (CTU) grant No. SGS26/184/OHK3/3T/18 “Intelligent, safe, and secure embedded systems”.

References

- [Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th ACM STOC*, pages 99–108. ACM Press, May 1996. doi:10.1145/237814.237838.
- [BHMS84] Robert K. Brayton, Gary D. Hachtel, Curtis T. McMullen, and Alberto L. Sangiovanni-Vincentelli. *Logic Minimization Algorithms for VLSI Synthesis*. The Kluwer International Series in Engineering and Computer Science. Kluwer Academic Publishers, Boston, MA, USA, 1984. doi:10.1007/978-1-4613-2821-6.
- [BM10] Robert K. Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In Tayssir Touili, Byron Cook, and Paul B. Jackson, editors, *Computer Aided Verification*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010. doi:10.1007/978-3-642-14295-6_5.
- [dD19] Florent de Dinechin. Reflections on 10 years of flopoco. In Naofumi Takagi, Sylvie Boldo, and Martin Langhammer, editors, *26th IEEE Symposium on Computer Arithmetic, ARITH 2019, Kyoto, Japan, June 10-12, 2019*, pages 187–189. IEEE, 2019. doi:10.1109/ARITH.2019.00042.
- [dDP10] Florent de Dinechin and Bogdan Pasca. Floating-point exponential functions for dsp-enabled fpgas, 2010. doi:10.1109/FPT.2010.5681764.

- [DP15] Léo Ducas and Thomas Prest. Fast Fourier orthogonalization. Cryptology ePrint Archive, Report 2015/1014, 2015. URL: <https://eprint.iacr.org/2015/1014>.
- [GGH97] Oded Goldreich, Shafi Goldwasser, and Shai Halevi. Public-key cryptosystems from lattice reduction problems. In Burton S. Kaliski, Jr., editor, *CRYPTO'97*, volume 1294 of *LNCS*, pages 112–131. Springer, Berlin, Heidelberg, August 1997. doi:10.1007/BFb0052231.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions, May 2008. doi:10.1145/1374376.1374407.
- [HHP⁺03] Jeffrey Hoffstein, Nick Howgrave-Graham, Jill Pipher, Joseph H. Silverman, and William Whyte. NTRUSIGN: Digital signatures using the NTRU lattice. In Marc Joye, editor, *CT-RSA 2003*, volume 2612 of *LNCS*, pages 122–140. Springer, Berlin, Heidelberg, April 2003. doi:10.1007/3-540-36563-X_9.
- [IEE19] IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019, 2019. doi:10.1109/IEEESTD.2019.8766229.
- [KA23] Emre Karabulut and Aydin Aysu. A hardware-software co-design for the discrete gaussian sampling of FALCON digital signature. Cryptology ePrint Archive, Report 2023/908, 2023. URL: <https://eprint.iacr.org/2023/908>.
- [Kar19] Angshuman Karmakar. const_gauss_split. GitHub repository, 2019. Commit ff1b170f903024b9c8e9dcf8b09d3afd93056609; accessed 2025-04-15. URL: https://github.com/Angshumank/const_gauss_split.
- [Kle00] Philip N. Klein. Finding the closest lattice vector when it's unusually close. In David B. Shmoys, editor, *11th SODA*, pages 937–941. ACM-SIAM, January 2000.
- [KRVV19] Angshuman Karmakar, Sujoy Sinha Roy, Frederik Vercauteren, and Ingrid Verbauwhede. Pushing the speed limit of constant-time discrete Gaussian sampling. A case study on Falcon. Cryptology ePrint Archive, Report 2019/267, 2019. URL: <https://eprint.iacr.org/2019/267>.
- [KY76] Donald E. Knuth and Andrew C.-C. Yao. The complexity of nonuniform random number generation. In Joseph F. Traub, editor, *Algorithms and Complexity: New Directions and Recent Results*, pages 357–428. Academic Press, New York, NY, USA, 1976.
- [LG20] Michael X. Lyons and Kris Gaj. Sampling from discrete distributions in combinational hardware with application to post-quantum cryptography. In *2020 Design, Automation & Test in Europe Conference & Exhibition, DATE 2020, Grenoble, France, March 9-13, 2020*, pages 610–613. IEEE, 2020. doi:10.23919/DATE48585.2020.9116434.
- [LYN⁺24] Yongseok Lee, Jonghee M. Youn, Kevin Nam, Heon Hui Jung, Myunghyun Cho, Jimyung Na, Jong-Yeon Park, Seungsu Jeon, Bo Gyeong Kang, Hyunyoung Oh, and Yunheung Paek. An efficient hardware/software co-design for FALCON on low-end embedded systems. *IEEE Access*, 12:57947–57958, 2024. doi:10.1109/ACCESS.2024.3387489.

- [MMW⁺25] Yukio Miyasaka, Alan Mishchenko, John Wawrzynek, Dino Ruic, and Xiaoqing Xu. High-effort logic synthesis using randomized transduction. In Yuichi Nakamura and Yu Wang, editors, *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, pages 58–64. ACM, 2025. doi:10.1145/3658617.3697720.
- [MPR⁺24] Dustin Moody, Ray Perlner, Andrew Regenscheid, Angela Robinson, and David Cooper. Transition to post-quantum cryptography standards. NIST Internal Report NIST IR 8547 ipd, National Institute of Standards and Technology, Gaithersburg, MD, November 2024. Initial Public Draft. doi:10.6028/NIST.IR.8547.ipd.
- [Nat17] National Institute of Standards and Technology. Post-quantum cryptography standardization process. Computer Security Resource Center project page, 2017. Continuously updated; accessed 2026-07-10. URL: <https://csrc.nist.gov/pqc-standardization>.
- [NR06] Phong Q. Nguyen and Oded Regev. Learning a parallelepiped: Cryptanalysis of GGH and NTRU signatures. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 271–288. Springer, Berlin, Heidelberg, May / June 2006. doi:10.1007/11761679_17.
- [OZZ⁺25] Yi Ouyang, Yihong Zhu, Wenping Zhu, Bohan Yang, Zirui Zhang, Hanning Wang, Qichao Tao, Min Zhu, Shaojun Wei, and Leibo Liu. FalconSign: An efficient and high-throughput hardware architecture for falcon signature generation. *IACR TCHES*, 2025(1):203–226, 2025. doi:10.46586/tches.v2025.i1.203-226.
- [PFH⁺22] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [PMKA25] Sharath Pendyala, Rahul Magesh, Elif Bilge Kavun, and Aydin Aysu. Out-running the millennium FALCON: Speed records for FALCON on xilinx FPGAs. Cryptology ePrint Archive, Report 2025/1490, 2025. URL: <https://eprint.iacr.org/2025/1490>.
- [PRR19] Thomas Prest, Thomas Ricosset, and Mélissa Rossi. Simple, fast and constant-time gaussian sampling over the integers for Falcon. In *Second NIST Post-Quantum Cryptography Standardization Conference*. National Institute of Standards and Technology, August 2019. URL: <https://csrc.nist.gov/CSRC/media/Events/Second-PQC-Standardization-Conference/documents/accepted-papers/rossi-simple-fast-constant.pdf>.
- [RC04] Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer Texts in Statistics. Springer, New York, NY, USA, 2 edition, 2004. doi:10.1007/978-1-4757-4145-2.
- [SAW⁺23] Michael Schmid, Dorian Amiet, Jan Wendler, Paul Zbinden, and Tao Wei. Falcon takes off - A hardware implementation of the falcon signature scheme. Cryptology ePrint Archive, Report 2023/1885, 2023. URL: <https://eprint.iacr.org/2023/1885>.

- [Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th FOCS*, pages 124–134. IEEE Computer Society Press, November 1994. doi:10.1109/SFCS.1994.365700.
- [SKD20] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. Cryptology ePrint Archive, Report 2020/071, 2020. URL: <https://eprint.iacr.org/2020/071>.
- [Voj25] Michal Vojáček. Efficient Falcon signature generation in hardware. Bachelor's thesis, Czech Technical University in Prague, May 2025. URL: <https://hdl.handle.net/10467/123977>.
- [YSZ⁺24] Xinglong Yu, Yi Sun, Yifan Zhao, Honglin Kuang, and Jun Han. RVCE-FAL: A RISC-V scalar-vector custom extension for faster FALCON digital signature. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2024, Valencia, Spain, March 25-27, 2024*, pages 1–6. IEEE, 2024. doi:10.23919/DATE58400.2024.10546713.
- [ZASM25] Binke Zhao, Ghada Alsuhli, Hani H. Saleh, and Baker Mohammad. Bi-samplerz: A hardware-efficient gaussian sampler architecture for quantum-resistant falcon signatures. *CoRR*, abs/2505.24509, 2025. URL: <https://doi.org/10.48550/arXiv.2505.24509>, arXiv:2505.24509, doi:10.48550/ARXIV.2505.24509.